Graphic Era deemed to be University DEHRADUN

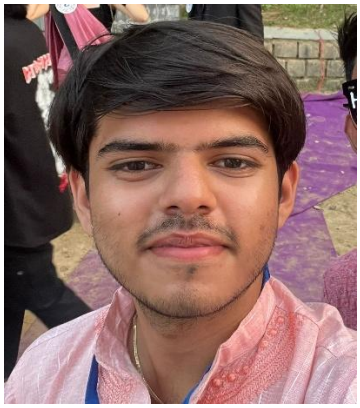
PROJECT AND TEAM INFORMATION

Project Title

ChargeX - Electric Vehicle Transport Management System

Student / Team Information

Team Name: Team # (Mentor needs to assign)	**ChargeX** **FS-VI-138**
Team member (Team Lead)	Tiwari, Pranav:230211301 pranavpiyush21111@gmail.com
Team member 2	Prajapati, Akshat – 230211153 Akshat.prj@gmail.com

Team member 3	Purohit,Nakshatra-230211286 Nakshatrapurohit27@gmail.com 
Team member 4	Rawat, Om : 23021808 Omrajput641@gmail.com

PROPOSAL DESCRIPTION

Motivation

The current roadblock to Broad Adoption of **Electric Vehicles (EV)** are the short distances they can go on a single charge, or "range anxiety", and also the uncertainty of where EV owners can charge their vehicles as there are currently limited locations with publicly accessible charging stations. While it is possible to find a mapping software that could give you a basic idea of how far you could travel on your EV's battery, EV owners will benefit from a comprehensive approach to finding an available charging station with a Full Stack Web Solution that allows for real-time time-slot availability, the safe booking of a charging station and management of your charging station all in one place. The goal of this project is to develop a fully functional and responsive web application that can provide benefits to EV owners and charging station providers through the use of today's most advanced web technology.

State of the Art / Current solution

As electric vehicles continue to grow in popularity, the need for efficient management of EV charging infrastructure is becoming increasingly important as well. Present-day systems, such as ChargePoint, EVgo, PlugShare, and Tata Power EZ Charge, provide end-users with the capability to locate charging stations, monitor their charging activity, and manage their accounts; however, they are primarily geared toward end-users and do not provide efficient functionality for administrators or station managers.

The available systems suffer from poor slot booking systems; therefore, users frequently experience congestion, extended wait times to use charging equipment, and inefficient utilization of charging infrastructure. Additionally, there are no comprehensive dashboards available to allow end-users or management to track their charging activity, energy usage, or system performance.

Another limitation associated with the current EV charging systems is the lack of integrated management; therefore, station management and user management, as well as analytics, are managed independently of each other, leading to an inefficient management system.

Currently, many of the existing EV charging systems are in the process of migrating to cloud-based solutions with REST APIs, secure user authentication, and real-time database capabilities providing for more efficient data management and management of the EV charging infrastructure. Additionally, many systems are beginning to incorporate dashboard-style management systems that facilitate improved infrastructure management and decision-making.

ChargeX supports the shortcomings of the current systems by giving an integrated approach that includes separate dashboards based on user role (admin, station manager, user) to facilitate effective booking of slots when charging for electric vehicles ("EV") within your office building's charging stations; the tracking of EV charging sessions; real time analytic; centralized station management leveraging a scalable architecture developed on cloud technology

Project Goals and Milestones

The purpose of ChargeX is to create a comprehensive Smart EV Charging Station Management System that allows for smooth interaction among station administrators, managers, and EV owners.

- **Functionality:** Allow users to find charging stations, reserve time slots to charge their cars, manage their charging sessions, and provide feedback.
- **Management:** Allow station administrators and managers to oversee station performance, view real-time data, and manage user and vehicle data.
- **Technical Excellence:** Use the latest technology stack (FastAPI, React, and Supabase) that provides a scalable, secure (RBAC and JWT) and world-class user experience.

Milestones and Roadmap

- **Core Systems -** Multi-Role Authentication, Station Management, Booking System and Charging Sessions.
- **UI/UX -** A web-responsive React front-end with Admin/Manager/User dashboards.
- **Backend -** A RESTful API using FastAPI with Supabase for RLS security.
- **Analytics -** A real-time consumption reporting and feedback system.
- **Notification -** Timely notifications of booking confirmations and session changes.
- **Payments -** Payment Gateway integration for sessions.
- **Advanced Analytics -** An advanced dashboard with detailed reporting.

Future

- **Infrastructure -** Docker - CI/CD & Automated testing.
- **Advanced Tech -** Microservices Architecture, ML Analytics for Predicted Usage Patterns & IoT/Blockchain Integrated Systems.

Project Approach

ChargeX has a Three-Tier Architecture that ensures a separation of concerns:

1. **Frontend Layer:** React Single Page Application (SPA) for a responsive and modern user interface.

2. **API Layer:** FastAPI (Python) backend providing a stateless, RESTful API.

3. **Data Layer:** Supabase PostgreSQL for robust data persistence and real-time functionality.

Technical Strategy

- **Modern & Scalable:** Utilizes component-based UI development (React+Vite) and high-performance async backend (FastAPI+Uvicorn).

- **Security First:** Implements JWT Authentication and Role-Based Access Control (RBAC) at the API level, supplemented by Row Level Security (RLS) directly in the database.

Development Workflow

- **Code Quality:** Strict adherence to PEP 8 (Python) and ES6+ (JavaScript/React) coding standards.

- **Testing:** Mandatory testing with pytest (Backend) and npm test (Frontend), targeting >80% code coverage.

- **Contribution Process:** Follows a workflow using Feature Branches (feature,fix) and Conventional Commits for a clean commit history.

- **Documentation:** Commitment to thorough documentation (API, Architecture, Setup) that stays up-to-date with code changes.

Deployment Strategy

- **Current:** Local development environment with a separate frontend and backend server.

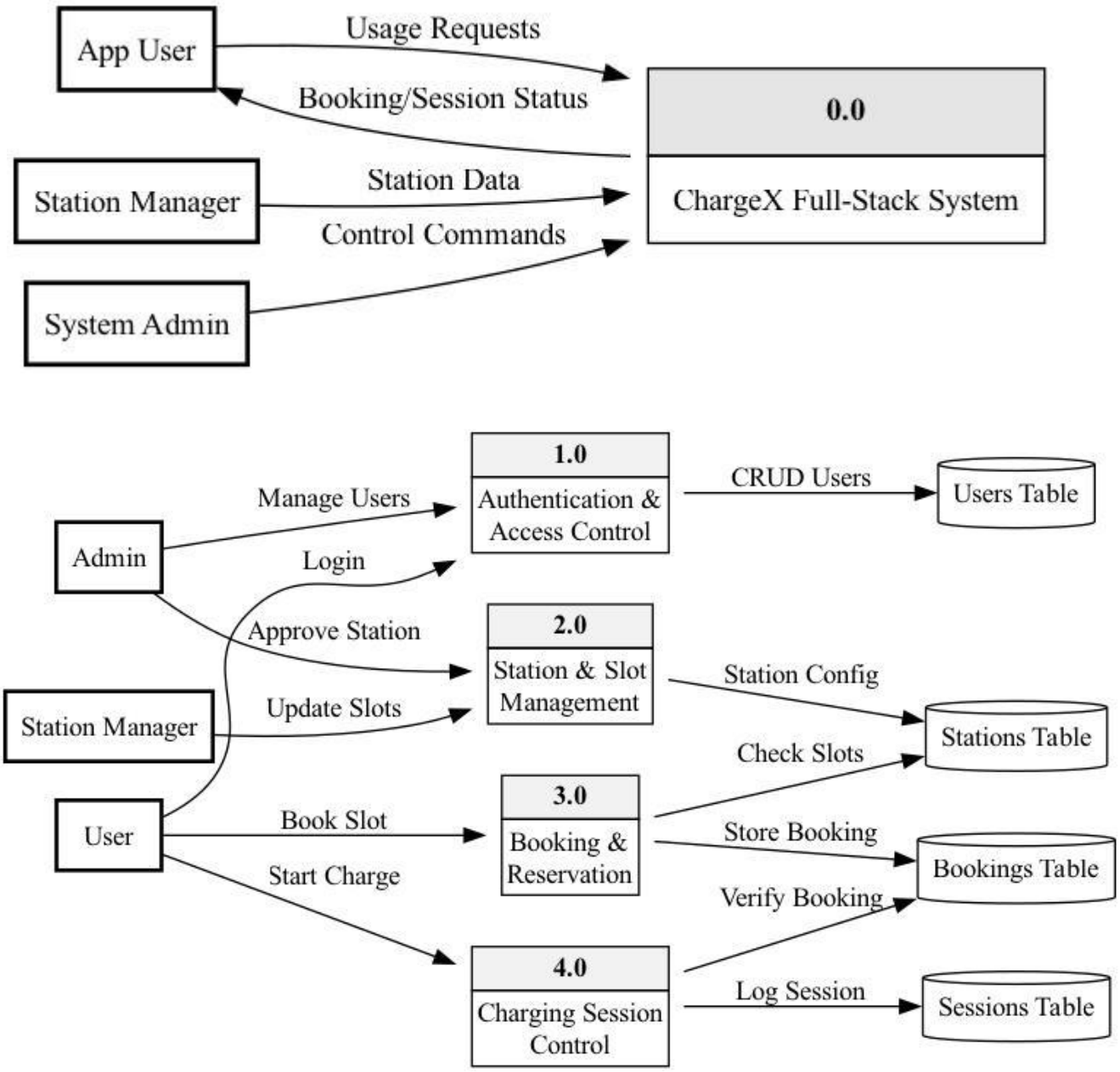
- **Production:** Static hosting for Frontend, separate server/container for Backend, and a managed Supabase instance for Database.

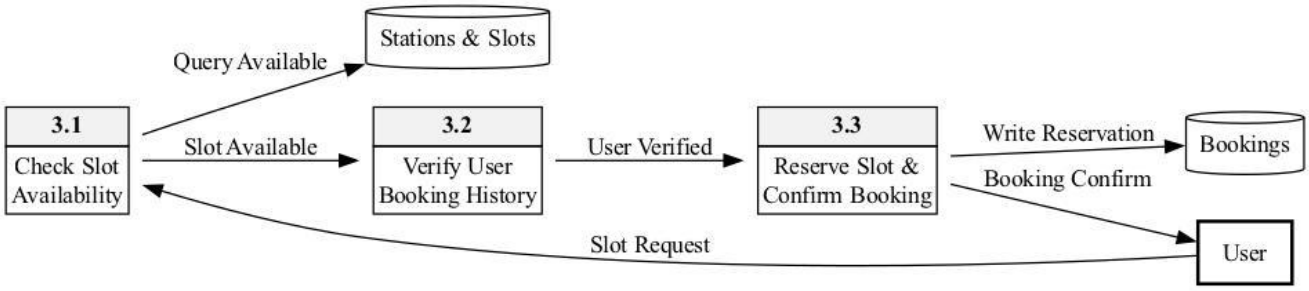
- **Future:** Containerization (Docker) and Microservices architecture for better scalability.

System Architecture (High Level Diagram)

A current Client-Server Architecture model, together with a Backend-as-a-Service (BaaS) provider, comprises the application:

Frontend (Client): The application is a React19 single-page application (SPA) that talks to the backend via RESTful APIs and directly to Supabase for instant subscriptions.
Backend (API Layer): FastAPI service handles complex business logic (booking validation, session handling), doing admin work and integrating data.
Database Layer: Supabase Postgres is the single source of truth that enforces data integrity and security through RLS policies.





Project Outcome / Deliverables

- **Web Application**: A responsive Single Page Application (SPA) built with **React 19** and Tailwind CSS, featuring dedicated dashboards for Admin, Manager, and User roles.
- **API Server**: A high-performance **FastAPI** backend (v1.0.0) exposing RESTful endpoints for authentication, station management, and booking operations.
- **Database**: A **Supabase PostgreSQL** schema with configured tables, indexes, and Row Level Security (RLS) policies.

Key Functionality

- **Role-Based Access Control:** Secure login and distinct permissions for Admins, Managers, and Users.
- **Station Management**: CRUD operations for charging stations and real-time status monitoring.
- **Booking System**: End-to-end flow for reserving charging slots and managing sessions.
- **Analytics Dashboard**: Visualized data on utilization, energy consumption, and user activity.

Assumptions

- **Admin Seeding**: The first Admin user is assumed to be technical enough to execute raw SQL queries (`INSERT INTO admins..) via the Supabase dashboard to elevate privileges.`
- **Device Usage**: Users (Admins, Managers, EV Owners) are assumed to access the platform via modern web browsers on desktop or mobile devices (Responsive Web App).
- **Single-Tenant Focus**: The current architecture assumes a single deployment per organization (ChargeX), rather than a multi-tenant SaaS serving multiple competing providers on one instance.

References

This project is built on the following technologies; for implementation, please use the documentation on the respective technologies as the source reference.

Backend & Data:

FastAPI: fastapi.tiangolo.com (Web Framework)

Supabase: supabase.com/docs (Backend-as-a-Service & Database)

PostgreSQL: postgresql.org/docs (Relational Database Engine)

Pydantic: docs.pydantic.dev (Data Validation)

Frontend & UI:

React 19: react.dev (UI Library)

Vite: vitejs.dev (Build Tool)

Tailwind CSS: tailwindcss.com (Styling Framework)

Lucide React: lucide.dev (Iconography)

The Following Technical Standards Are In Place:

RESTful API: The standard architectural style for web APIs is documented in the API section docs/API.md.

JWT (JSON Web Token): Standard RFC 7519 used for secure user authentication.

Semantic Versioning: Versioning scheme (i.e., v1.0.0) used for release management. Please see docs/CHANGE.